

SQL Data Warehouse Project Report

Author: Sashi Preetham | **Database:** PostgreSQL | **Architecture:** Medallion (Bronze → Silver → Gold)

1. Executive Summary

This project presents the end-to-end design and implementation of a SQL Data Warehouse using PostgreSQL. Raw data from CRM and ERP source systems was ingested, cleansed, and transformed into a star schema analytical model ready for Power BI reporting.

The solution addresses common data engineering challenges such as duplicate records, inconsistent formats, missing values, and multi-source data integration — delivering a single trusted source of truth for business decision-making.

2. Business Problem

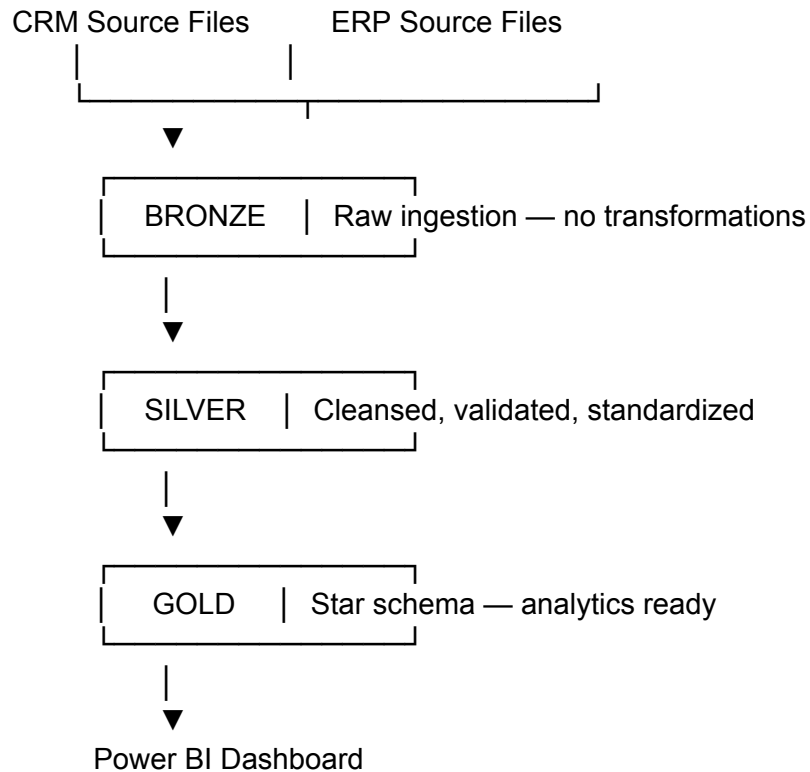
Organizations operating multiple source systems often face:

Challenge	Impact
Duplicate & inconsistent records	Unreliable reporting
Missing or null values	Incomplete analysis
Varying data formats across systems	Difficult data integration
No single source of truth	Poor decision-making

Goal: Build a scalable, layered data warehouse that consolidates, cleanses, and models data for reliable analytics.

3. Architecture

The project follows the **Medallion Architecture** — a three-layer design pattern widely adopted in modern data engineering.



4. Technology Stack

Component	Tool
Database	PostgreSQL
Query Language	SQL
Reporting	Power BI
Administration	pgAdmin 4
Version Control	Git & GitHub
Documentation	Notion

5. Source Data

CRM System

File	Contents
cust_info.csv	Customer master records
prd_info.csv	Product catalog
sales_details.csv	Sales transactions

ERP System

File	Contents
cust_az12.csv	Customer demographics (age, gender)
loc_a101.csv	Customer location and country
px_cat_g1v2.csv	Product categories and subcategories

6. Layer-by-Layer Implementation

Bronze Layer — Raw Ingestion

- Loaded all 6 source CSV files as-is into PostgreSQL schemas
 - No transformations applied — data preserved exactly as received
 - Serves as the historical audit trail and reprocessing baseline
-

Silver Layer — Cleansing & Standardization

Customer Data

Issue

Fix Applied

Duplicate customer records	<code>ROW_NUMBER()</code> — kept latest by <code>cst_create_date</code>
Inconsistent gender values	<code>'M' → 'Male', 'F' → 'Female', else 'n/a'</code>
Inconsistent marital status	<code>'M' → 'Married', 'S' → 'Single', else 'n/a'</code>
Leading/trailing whitespace	<code>TRIM()</code> applied to all name fields

Product Data

Issue	Fix Applied
Category ID embedded in product key	Extracted using <code>SUBSTRING()</code> and <code>REPLACE()</code>
Abbreviated product line codes	<code>'M' → 'Mountain', 'R' → 'Road', etc.</code>
NULL product costs	Replaced with <code>0</code> using <code>COALESCE()</code>
Missing product end dates	Derived using <code>LEAD()</code> window function (SCD Type 2)

Sales Data

Issue	Fix Applied
Dates stored as INT (YYYYMMDD)	Converted using <code>TO_DATE()</code>
Invalid/zero dates	Set to <code>NULL</code>
Incorrect sales amounts	Recalculated as <code>quantity × ABS(price)</code>
NULL or negative prices	Derived as <code>sales / NULLIF(quantity, 0)</code>

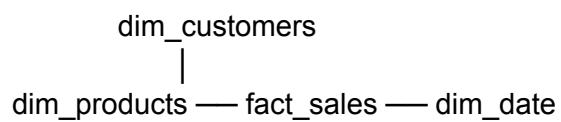
ERP Data

Issue	Fix Applied
<code>'NAS'</code> prefix in customer IDs	Stripped using <code>SUBSTRING()</code>

Future birth dates	Set to NULL
Abbreviated country codes	'DE' → 'Germany', 'US' / 'USA' → 'United States'
Inconsistent gender values	Normalized to 'Male' / 'Female' / 'n/a'

🏆 Gold Layer — Analytical Model

The Gold layer implements a **Star Schema** optimized for reporting.



dim_customers

Integrates CRM customer data with ERP demographics and location into a single customer view.

Key fields: **customer_key**, **customer_id**, **full_name**, **gender**, **marital_status**, **country**, **birth_date**

dim_products

Combines CRM product data with ERP category information.

Key fields: **product_key**, **product_id**, **product_name**, **category**, **subcategory**, **product_line**, **product_cost**

fact_sales

Central transaction table joined to all dimensions.

Key fields: **order_number**, **product_key**, **customer_key**, **order_date**, **sales_amount**, **quantity**, **price**

7. Data Quality Checks

Quality validation was performed at both Silver and Gold layers.

Silver Layer

-- Duplicate check

```
SELECT cst_id, COUNT(*) FROM silver.crm_cust_info GROUP BY cst_id HAVING COUNT(*) > 1;
```

-- Null check

```
SELECT * FROM silver.crm_cust_info WHERE cst_id IS NULL;
```

-- Standardization check

```
SELECT DISTINCT cst_gndr FROM silver.crm_cust_info;
```

Gold Layer

-- Referential integrity check

```
SELECT * FROM gold.fact_sales  
WHERE customer_key NOT IN (SELECT customer_key FROM gold.dim_customers);
```

8. Key SQL Techniques

Technique	Used For
ROW_NUMBER()	Deduplication
LEAD()	SCD Type 2 end date derivation
COALESCE()	NULL handling
CASE WHEN	Value standardization
TRIM() / UPPER()	String cleansing
TO_DATE()	Date format conversion
CTEs	Readable multi-step transformations
JOINS	Cross-source data integration
Views	Gold layer analytical models

9. Results & Outcomes

-  Consolidated 6 source files from 2 systems into a unified warehouse
 -  Eliminated duplicate and inconsistent records across all tables
 -  Built a star schema model ready for Power BI reporting
 -  Established reusable data quality validation queries
 -  Delivered clean, trusted data for business decision-making
-

10. Future Enhancements

- Incremental loading to replace full truncate-and-reload
 - Automated ETL scheduling
 - Data lineage and audit logging
 - Cloud deployment on AWS RDS or Azure PostgreSQL
 - Advanced Power BI dashboard with KPIs and drill-throughs
-

11. Conclusion

This project demonstrates the complete lifecycle of a modern data warehouse — from raw ingestion through cleansing to analytical modeling. The Medallion Architecture provides a scalable, maintainable foundation, and the star schema delivers a reporting layer that is both performant and easy for business users to consume.
